

Phase 1 — Conception

Hotel Booking Chatbot · **Birol Hotel**
IU International University of Applied Sciences
Course: Project — AI Use Case (CSEMAIPAIUC01) · Task 1
Student: **Birol, Egemen** · Matriculation 12345678 · Berlin

1. Value proposition

Following the lecture's value-proposition canvas (Lec 01) the project is positioned as follows:

For leisure and business travellers (the minimum viable segment) who want to book a hotel room in under a minute,
Dissatisfied with long multi-step web forms that demand account creation, hide live availability behind filters, and behave poorly on phones,
Due to the unmet need for a fast, mobile-friendly, conversational booking flow that combines reservation, hotel info and Istanbul tourism advice in one channel,
Birol Hotel offers **Aria**, a conversational booking concierge,
That provides a single-channel chat interface, instant confirmation with a unique reference, transparent pricing, in-bot card capture, session-isolated receipts, a CEO occupancy dashboard, and 24/7 availability — all without account creation.

2. Aims, target audience and MVP scope

Aims. Aria's primary aim is to capture the minimum information required to create a valid room reservation through natural-language dialogue (per the assignment: name, time period, number of guests are mandatory; payment and breakfast are optional extensions). The secondary aim is to act as a 24/7 concierge, answering common questions about the hotel and Istanbul, and to offer a CEO-only occupancy view of the property.

Minimum Viable Segment (MVS, Lec 01). English-speaking leisure and business travellers, mostly on mobile ($\approx 70\%$ of online hotel bookings originate on phones). The same segment is reachable via a single channel: a website / WebView-wrapped iOS app distributed by URL.

Minimum Viable Product (MVP). A static web app + thin iOS shell that delivers, end-to-end, the smallest set of features that satisfy the core need: a guided booking dialogue (eight slots: name, dates, room_type, guests, breakfast, payment, payment_card, confirm), 15 concierge Q&A; intents, a soft-fall fallback for off-topic queries, a calendar widget with availability overlays, an in-bot card form, per-session receipt isolation, and a password-gated admin dashboard.

3. Structure and process rationale

A **slot-filling state machine** was chosen over a free-form LLM for three reasons. (1) *Correctness is paramount* in a booking system — a hallucinated reservation is unacceptable. (2) *Auditability*: the dialog graph is visible in source and trivially reviewable by a tutor or a QA engineer. (3) *Cost & privacy*: the project runs entirely client-side, so no PII ever leaves the device, and there is no per-message inference bill.

A **static-site architecture** was chosen over Rasa or Dialogflow because the assignment grading machine must be able to run the system with no setup — GitHub Pages eliminates the install step entirely, and the iOS app is a thin WKWebView wrapper that auto-updates whenever a commit lands on main (zero re-install for the user).

4. Frameworks, tools and trade-offs (Lec 01)

	Rasa	Dialogflow	Custom (chosen)
--	------	------------	-----------------

Hosting	Self-host server	Google Cloud	GitHub Pages (free, HTTPS)
Setup at grading time	Heavy install (broken on Py 3.14)	Google account + internet	Open the URL
NLU	Built-in pipeline (DIET, etc.)	Hosted neural model	Bag-of-words + cosine + regex
Dialog	Rule + ML stories	Contexts / fulfilment	Slot-filling FSM (8 slots)
Cost / mo	Server costs	Per-call API bill	€0 (Pages free tier)
Privacy	All on your server	PII to Google	All client-side, never leaves device
iOS path	Need backend + API	Need backend + API	Same site in WKWebView
Auditable	Yes	Partly (cloud config)	Yes — every line in repo

Decision: Custom static stack. The decisive factors are zero-setup for the grader, full client-side privacy, and the auto-update path for the iOS app (push to GitHub → app reloads on launch). The trade-off is weaker semantic generalisation than a neural NLU; we accept this because the booking domain is closed and a paraphrase that misses our pattern library hits a graceful soft-fall fallback chip menu.

5. Information captured

Mandatory (assignment spec): guest name · stay period (check-in & check-out) · number of guests.

Optional extensions implemented: room type (Standard / Deluxe / King Suite / Bosphorus Suite — €120 / €180 / €280 / €420 per night), breakfast preference (+€10 / guest / night), payment method (credit · debit · pay-at-hotel), and a glass card-form interlude for credit/debit with a Pay-€X button that performs no real charge.

6. UML — Use case diagram

Use cases are grouped by colour: **concierge** (warm cream), **booking** (teal), **admin** (rose). Guest is the primary actor for booking and concierge use cases; CEO is the primary actor for the admin dashboard. Browser storage is a secondary actor that persists confirmed reservations (*localStorage*) and the per-guest receipt scope (*sessionStorage*).

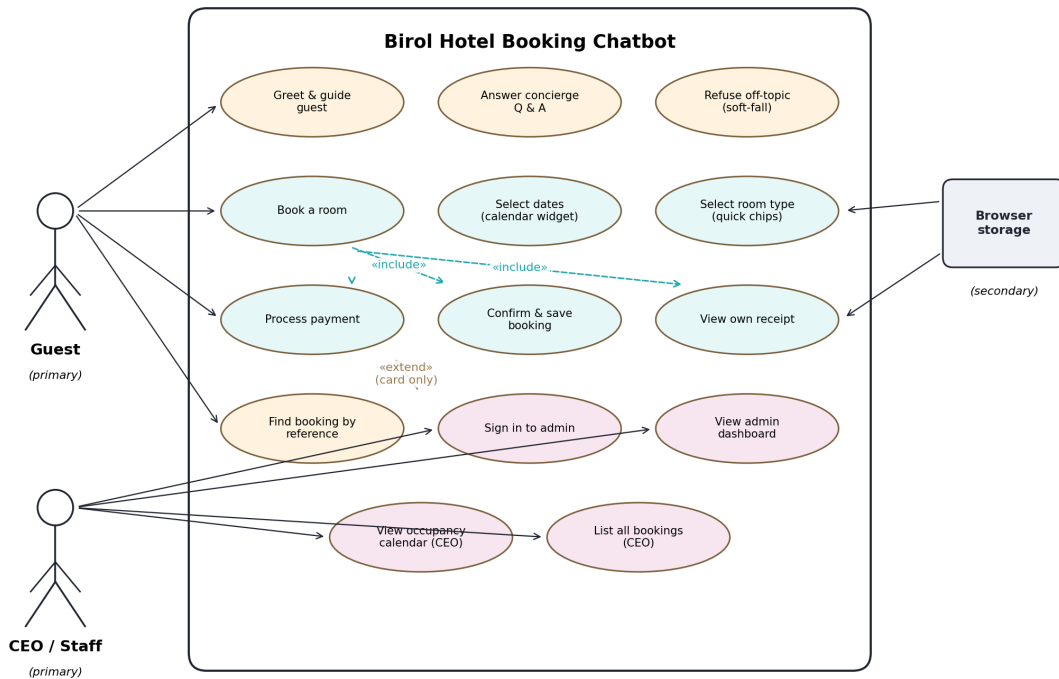


Figure 1 — Use case diagram. Includes 14 use cases across three swim-lanes (concierge / booking / admin). «include» and «extend» relationships drawn per Lec 02.

7. UML — Sequence diagram (booking happy path)

The lifelines model the runtime topology: the **Web UI** (chat.js + chat.html) talks to the **Aria Bot** (bot.js's HotelBookingBot.respond), which in turn calls the **Intent Classifier** and the **Entity Extractors**. The **Booking Store** writes to localStorage on confirmation. The two interactive widgets (calendar and card form) are rendered by the Web UI in response to slot transitions returned by the bot.

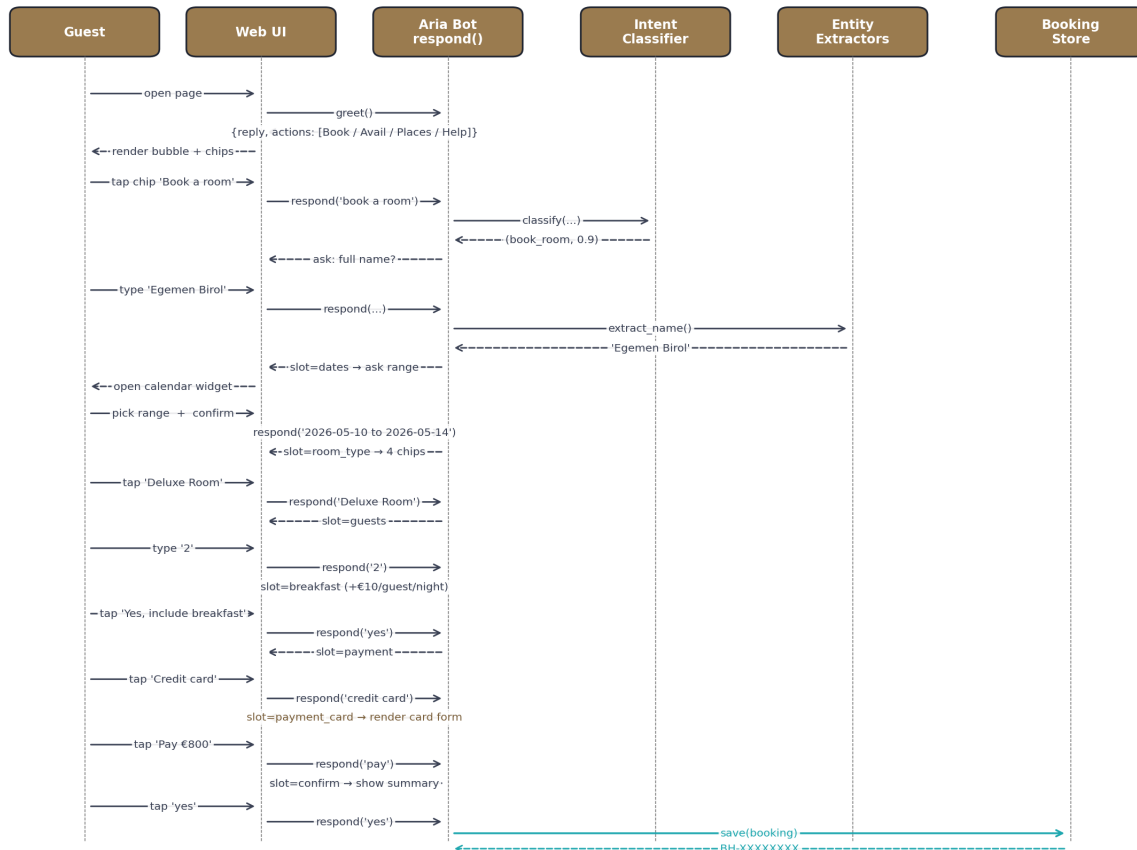


Figure 2 — Sequence diagram for the booking happy path, including the calendar widget and the new card-payment interlude.

8. UML — State machine (slot machine)

The dialog manager is a finite-state machine. Each slot has a deterministic transition function: a slot is filled by an entity extractor, the FSM advances to the next-missing slot. Mandatory slots are highlighted in brass (per the assignment spec). Concierge intents are handled *transversally* — they answer the question and then re-prompt the current slot, so a user can ask *where should I eat?* mid-booking without losing their place. The card form is the new `payment_card` state; pay-at-hotel skips it entirely.

Concierge intents (places to visit, wifi, parking, ...) are answered transversally and the bot returns to the current slot.

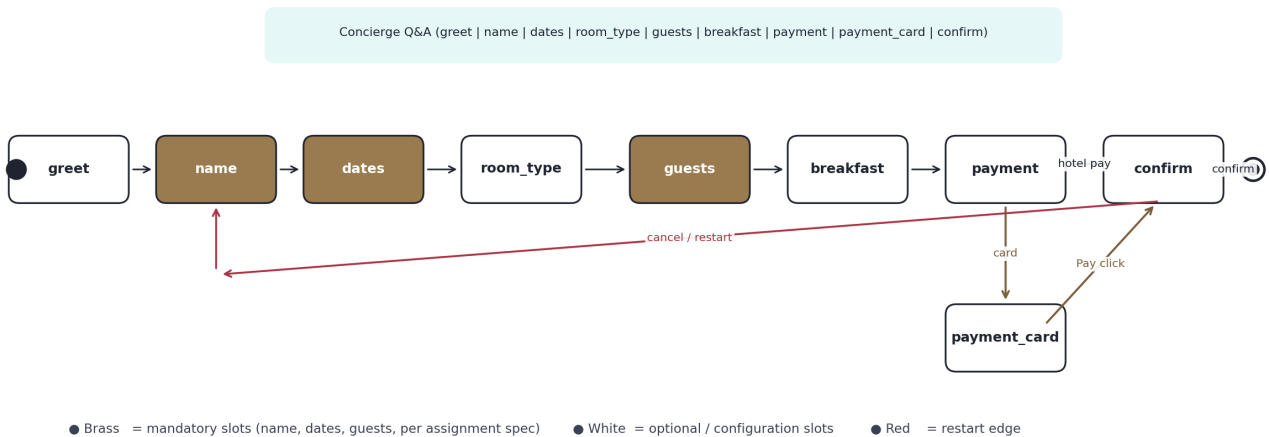


Figure 3 — State machine for the conversation flow.

9. UML — Component & deployment

The runtime is a single web container (the browser, or a WKWebView in the iOS app). All logic and data live inside that container. The deployment loop is push → Pages CDN rebuild → next-launch refresh — no servers, no build step, no native binary update.

Figure 4 — Component & deployment diagram

Static-site architecture — auto-deploys from GitHub on every push.

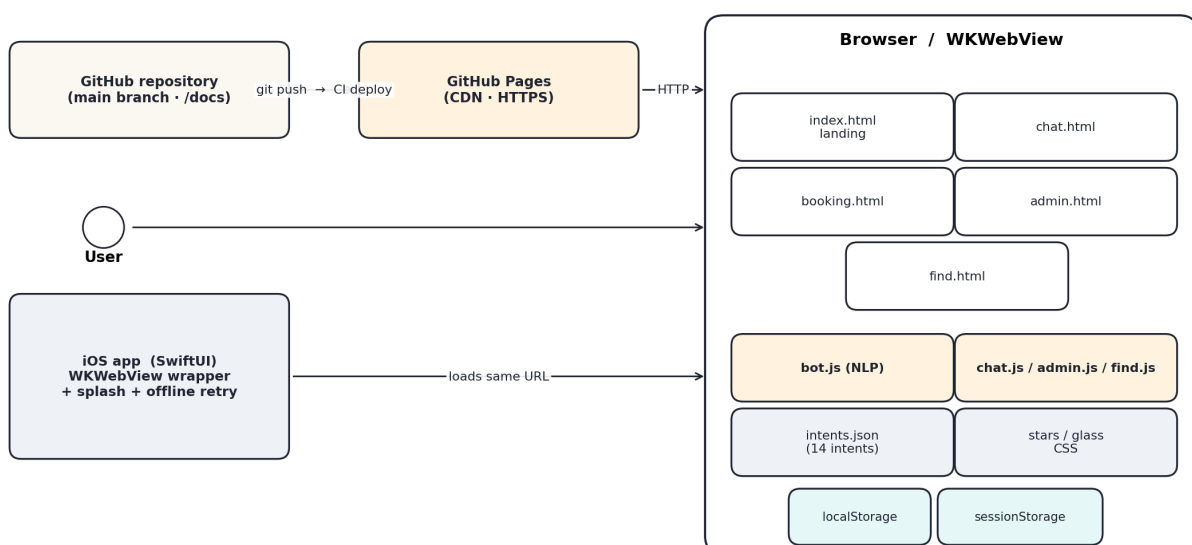


Figure 4 — Component & deployment diagram.

10. Chatbot development workflow (Lec 03/04)

The classical chatbot pipeline (define scope → intents → training data → conversation flow → train/test → iterate → deploy) maps directly onto the Rasa workflow taught in lecture, even though the prototype hand-implements each stage in plain JS. Below is the version that materialised in this project:

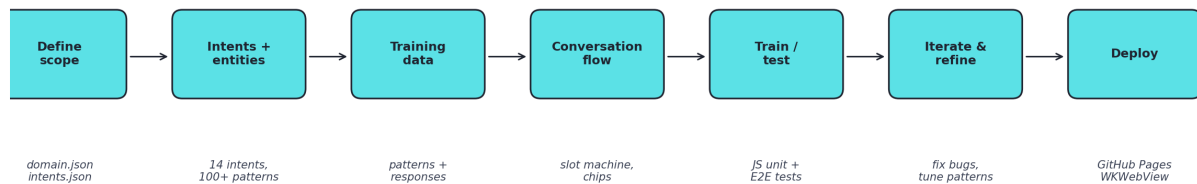


Figure 5 — Project pipeline. Each stage produced an artifact in the repo (domain via intents.json, training data via patterns, conversation flow via the slot machine in chatbot.py / bot.js, tests in test_bot.py and test_web_bot.js, deploy via GitHub Pages). 35 web tests + 17 Python tests passing at the time of writing.

11. Data & knowledge model

Knowledge base — intents.json (14 intents, ~120 patterns): greet, book_room, provide_name, provide_dates, provide_guests, breakfast_yes, breakfast_no, provide_payment, confirm_yes, confirm_no, goodbye, thanks, help, fallback + 16 concierge intents (places_to_visit, food_recommendations, transport_airport, check_in_time, breakfast_hours, wifi, parking, amenities, cancellation_policy, pets, currency_info, tipping, language, hotel_contact, neighborhood, emergency, room_types).

Domain entities: Booking{name, checkin, checkout, roomType, roomRate, guests, breakfast, payment, paymentConfirmed, booking_id, status, total_eur} · RoomType{id, name, rate, blurb} · Availability(date) → {free | peak-full | guest-booked}.

Persistence layers: localStorage holds confirmed bookings (global to the device, used by the CEO admin); sessionStorage holds the current guest's booking IDs so booking.html only shows that guest's reservation. CEO password is stored in sessionStorage with key birol.admin.session.

12. Workplan and risk register

Stage	Output	Status
Conception (this document)	1-page+ concept · UML diagrams · framework choice	this submission
Development	Working chatbot · 14 intents · slot FSM · UI · iOS shell	complete
Reflection	10-slide presentation · hyperlinks · screenshots	in progress
Finalization	2-page abstract · zip with all files · re-submitted Phase 1 + 2	pending feedback

Risk	Impact	Mitigation
Bag-of-words misses paraphrases	User feels unheard	Soft-fall fallback always offers Help + Book chips; ~30 patterns/intent on average
Stale browser cache after deploy	Users see old UI	?v= query bust on every asset · meta no-cache on every HTML
Receipt visible to wrong user	Privacy issue	sessionStorage tag → booking.html only shows the current session's bookings
CEO password is client-side	Easy to bypass	Documented as a soft gate; for prototype assessment only

No backend → no global view of all bookings	Architecture limit	CEO sees only this-device bookings; future work could swap to Firestore
---	--------------------	---

13. AI ethics and sustainability

Privacy. No personal data leaves the user's device. There are no analytics, no third-party trackers, no server-side logs. All NLU runs in the browser; the chat transcript and the booking record are kept in localStorage. The CEO admin can be cleared by deleting that store.

Inclusivity. Quick-reply chips reduce typing for non-native English speakers; the iridescent design respects prefers-reduced-motion (shooting stars freeze for users who set the system flag). All key controls are reachable by tab and have explicit ARIA labels.

Transparency. The card-form is explicitly marked as a demo (‘ Demo only — your card is not actually charged.’). The bot openly refuses off-topic questions instead of hallucinating (‘I focus on Birol Hotel and Istanbul...’).

Sustainability. Zero-backend means zero idle servers. GitHub Pages is served from edge caches. Each page is < 50 kB before fonts, so the carbon footprint per session is negligible compared to a typical hotel website.

Submitted in fulfilment of the Conception phase per the portfolio brief (IU CSEMAIPAIUC01). The Development presentation (Phase 2) and final abstract + product zip (Phase 3) follow the same naming convention.